

부하 테스트

시나리오 1 — 수강신청 전체 플로우 (EnrollmentFlowSimulation)

개요

300명이 로그인부터 수강 변경까지 실제 사용자 여정 전체를 동시에 수행. 로그인 포함 체인 전체 응답시간과 HikariCP 커넥션 압박을 측정.

API 호출 순서

`before()` 에서 300명 계정 생성, Redis `enrollment:course:3=300`, `enrollment:course:4=300` 초기화 후 본 시나리오 실행.

1. `POST /api/v1/auth/login` — 개인 계정으로 로그인, `accessToken` 발급
2. `GET /api/v1/courses` — 강의 목록 조회
3. `POST /api/v1/enrollments` — UI/UX 강의(`course_id=3`) 수강신청
4. `GET /api/v1/enrollments/me` — 내 수강신청 조회 (Kafka 비동기 처리 3초 대기 후 `enrollmentId` 추출)
5. `DELETE /api/v1/enrollments/{enrollmentId}` — 수강신청 취소
6. `POST /api/v1/enrollments` — 수강 변경 (리더쉽, `course_id=4` 재신청)

단계별 통과 인원: 로그인(300) → 강의조회(300) → 수강신청(300) → 내신청조회(270) → 취소(178) → 수강변경(178)

취소·재신청이 178명인 이유: 3초 대기 후 `/enrollments/me` 조회 시 Kafka Consumer 처리가 아직 완료되지 않은 건은 `enrollmentId` 추출 실패 → 해당 사용자는 이후 단계 진행 불가.

측정 결과 (300명)

항목	수강신청 전체 플로우
전체 요청 수	1526 (KO=0)
평균 응답시간	1832ms
P50	1563ms
P75	2549ms

항목	수강신청 전체 플로우
P95	4636ms
P99	5511ms
Max	6272ms
처리량	109 req/s
에러율	0%

응답시간 분포: 800ms 미만 26% / 800~1200ms 12% / 1200ms 이상 62%

발견된 문제 — 동기 구간에 DB 쿼리 혼입

설계 의도(docs/[architecture.md](#) — 학생 수강신청 흐름 동기 구간)는 동기 구간을 Redis-only로 정의하고 있으나, 실제 구현된 수강신청(`enroll()`) 코드는 DB를 여러 번 조회한다.

단계	설계 (CLAUDE.md)	현재 코드
1	lock:course:{id} 확인 → Redis	학생 조회 → DB (항상)
2	시간표 체크 → Redis (캐시 없으면 DB)	강의 조회 → DB (항상)
3	학점 체크 → Redis (캐시 없으면 DB)	중복 수강 체크 → DB (항상)
4	DECR enrollment:course:{id} → Redis	대기자 NOTIFIED 체크 → DB (항상)
5	Kafka 발행	시간표 체크 → Redis (캐시 없으면 DB)
6	응답 반환	학점 체크 → Redis (캐시 없으면 DB)
7	-	DECR → Redis
8	-	Kafka 발행

설계: DB 0번 (캐시 미스 제외) / 현재 코드: DB 4번 (항상) + 캐시 미스 시 최대 2번 추가
300명 동시 요청 시 DB 쿼리 최대 1800회 발생 → HikariCP 풀(10개) 고갈 → 응답시간 증가. 이것이 P95 4636ms의 실제 원인.

엔드포인트별 P95 (Grafana)

엔드포인트	P95
<code>POST /api/v1/auth/login</code>	~1.4s
<code>POST /api/v1/enrollments</code>	~2.6s

엔드포인트	P95
DELETE /api/v1/enrollments/{id}	~5.0s

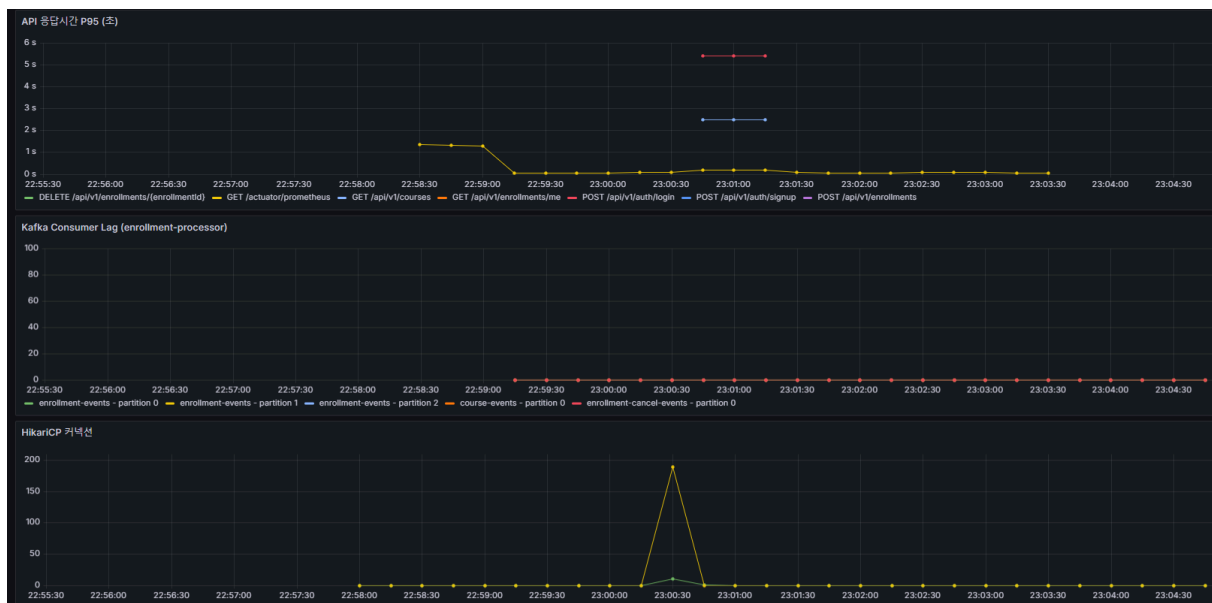
Grafana 병목 발견 (Before)

HikariCP 커넥션 대기 180개 급등: POST /api/v1/enrollments 및 GET /api/v1/enrollments/me 270명 동시 DB 조회 구간에서 대기 커넥션이 최대 180개까지 급등. 동기 구간 DB 쿼리 혼입 + 풀 사이즈 10개가 원인.

DELETE P95 5s: 취소 요청은 Cancel Consumer → RDS UPDATE → 대기자 알림까지 이어지는 체인 전체를 대기하므로 수강신청 대비 약 2배 느림.

Kafka Consumer Lag ≈ 0: enrollment-events, enrollment-cancel-events 모두 Lag 0 유지 → Kafka 자체는 병목 아님.

코드 수정 후 동일 시나리오 재측정하여 Before/After Grafana 스크린샷 비교 예정.



측정 결과 After — Redis 전환 후 (2026-06-19)

항목	Before (DB)	After (Redis)	개선
전체 요청 수	1526 (KO=0)	1407 (KO=0)	—
평균 응답시간	1832ms	1053ms	-43%
P50	1563ms	882ms	-44%
P75	2549ms	1804ms	-29%
P95	4636ms	2845ms	-39%
P99	5511ms	3518ms	-36%

Max	6272ms	4344ms	-31%
처리량	109 req/s	156.33 req/s	+43%
에러율	0%	0%	—

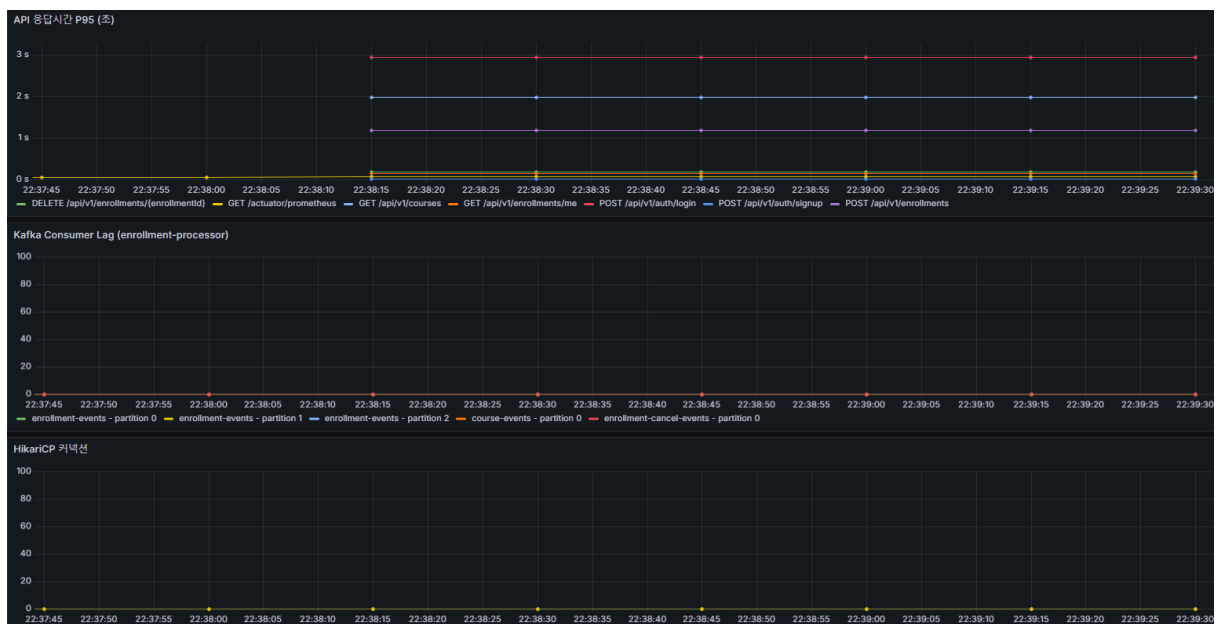
응답시간 분포: 800ms 미만 48.26% / 800~1200ms 10.52% / 1200ms 이상 41.22%

단계별 통과 인원: 로그인(300) → 강의조회(300) → 수강신청(300) → 내신청조회(169)
→ 취소(169) → 수강변경(169)

Grafana 병목 발견 (After)

HikariCP 대기 180 → ~0: enroll() 동기 구간 DB 쿼리 제거로 커넥션 풀 압박 해소.
Redis-only 전환 효과 확인.

Kafka Consumer Lag ≈ 0: enrollment-events, enrollment-cancel-events 모두
Lag 0 유지.



시나리오 2 — 대기자 전체 플로우 (WaitlistFlowSimulation)

개요

300명이 로그인부터 대기자 등록 → 취소 → 재등록까지 전체 플로우를 동시에 수행.
Redisson 분산 락 직렬화 응답시간 및 rank 정확도 측정.

API 호출 순서

`before()` 에서 300명 계정 생성, Redis `enrollment:course:3=0` (정원 소진), `waitlist:course:3=30` (대기 슬롯 30개) 초기화 후 본 시나리오 실행.

1. `POST /api/v1/auth/login` — 개인 계정으로 로그인, `accessToken` 발급
2. `GET /api/v1/courses` — 강의 목록 조회
3. `POST /api/v1/waitlists` — 대기자 등록 (30명 성공/201, 270명 마감/409 — KO=0으로 분류)
4. `DELETE /api/v1/waitlists/{waitlistId}` — 대기자 취소 (등록 성공 30명만)
5. `POST /api/v1/waitlists` — 대기자 재등록 (취소로 복구된 슬롯에 재등록)

측정 결과 Before — DB 직접 조회 (300명)

항목	대기자 전체 플로우
전체 요청 수	996 (KO=25)
평균 응답시간	1631ms
P50	1317ms
P75	2310ms
P95	3266ms
P99	4111ms
Max	4492ms
처리량	142.29 req/s
에러율	2.51% (25건)

응답시간 분포: 800ms 미만 14.66% / 800~1200ms 22.79% / 1200ms 이상 60.04% / 실패 2.51%

단계별 통과 인원: 로그인(300) → 강의조회(300) → 대기자등록(300) → 취소시도(48) → 재등록(48)

발견된 문제 — 동시 취소 시 DB 데드락

`WaitlistService.cancel()` 은 취소 후 `decrementRanksAfter(courseId, cancelledRank)` 를 호출해 이후 대기자 전원의 `rank`를 -1 업데이트한다. 48명이 동시에 취소하면 동일 행을 동시에 UPDATE하려다 MySQL 데드락 발생 → 트랜잭션 롤백 → HTTP 500 (25건).

WaitlistService 동기 구간 DB 직접 조회 현황 (EnrollmentService Redis 전환과 달리 미적용 상태):

단계	처리 방식
학생 조회	DB
강의 조회	DB
수강신청 중복 확인	DB
대기 중복 확인	DB
rank 계산 (Redisson 락 안)	DB
waitlist INSERT	DB

Grafana 병목 발견 (Before)

HikariCP 커넥션 대기 189: WaitlistService 동기 구간 전체 DB 직접 조회로 300명 동시 요청 시 커넥션 풀 포화.

DELETE /api/v1/waitlists/{waitlistId} P95 ~3s: 동시 취소 시 `decrementRanksAfter` 데드락으로 응답 지연.

Kafka Consumer Lag = 0: 대기자 등록은 Kafka 미사용 — 정상.



측정 결과 After — Redis 전환 후 (2026-06-20)

항목	Before (DB)	After (Redis)	변화
전체 요청 수	996 (KO=25)	998 (KO=27)	—
평균 응답시간	1631ms	1895ms	+16%
P50	1317ms	1503ms	+14%
P75	2310ms	2745ms	+19%

항목	Before (DB)	After (Redis)	변화
P95	3266ms	4060ms	+24%
P99	4111ms	4874ms	+19%
처리량	142.29 req/s	124.75 req/s	-12%
HikariCP 대기	189	80	-58%
에러율	2.51%	2.71%	—

결과 해석

register() 개선 확인: HikariCP 대기 189 → 80으로 58% 감소. Redis 전환으로 register() DB 쿼리 6개 → 1~2개 역할을 수치로 확인.

전체 P95는 오히려 악화: cancel()의 `decrementRanksAfter` 데드락이 여전히 전체 응답시간의 실제 병목. Redisson 락 제거로 cancel() DB 충돌이 오히려 심해짐.

포트폴리오 진단: register() 최적화는 DB 부하 감소에 효과적이었으나, 잘못된 병목 최적화는 전체 성능 개선으로 이어지지 않음. 진짜 병목은 cancel()의 `decrementRanksAfter` 데드락 - 이 문제가 해결될 때 After 개선 효과가 의미 있음.

Grafana (After)



에러 원인 — decrementRanksAfter 데드락

After 1에서 에러율이 Before(2.51%)보다 오히려 2.71%로 높아진 원인은 **cancel()**의 데드락 구조를 그대로 두었기 때문이다.

`cancel()` 호출 시 `decrementRanksAfter(courseId, cancelledRank)` — 취소된 순번보다 뒤에 있는 대기자 전원의 rank를 `UPDATE waitlist SET rank = rank - 1 WHERE rank > ?` bulk UPDATE — 가 핵심 문제다. 30명이 동시에 취소하면 같은 행을 동시에 수정하려다 **MySQL 데드락 발생** → **트랜잭션 롤백** → **HTTP 500**. `register()` 개선 후 Redisson 락이 제거되어 `cancel()` 동시 진입이 더 늘어나면서 데드락이 오히려 심해졌다.

해결 방법 — Redis Sorted Set (B안) 도입

`decrementRanksAfter()` DB bulk UPDATE를 완전히 제거하고 `waitlist:zset:course:{courseId}` Redis Sorted Set으로 순번 관리를 대체했다.

- **등록 시:** `ZADD waitlist:zset:course:{courseId} {현재시각ms} {waitlistId}` — score 오름차순 = 등록 순서 = 순번
- **순번 조회:** `ZRANK` 한 줄로 동적 계산 — 취소/만료가 반영된 항상 정확한 순번
- **취소/만료 시:** `ZREM` 한 줄로 제거 → 나머지 순번 자동 재계산
- bulk UPDATE 자체가 사라져 **데드락 구조 근본 제거**

측정 결과 After 2 — Redis Sorted Set 전환 (2026-06-21)

항목	Before (DB)	After 1 (DB → Redis)	After 2 (Sorted Set)
전체 요청 수	996 (KO=25)	998 (KO=27)	1052 (KO=0)
평균 응답시간	1631ms	1895ms	2020ms
P50	1317ms	1503ms	1577ms
P75	2310ms	2745ms	3066ms
P95	3266ms	4060ms	4693ms
P99	4111ms	4874ms	5827ms
처리량	142 req/s	124 req/s	116 req/s
HikariCP 대기	189	80	~0
에러율	2.51% (25건)	2.71% (27건)	0%

Grafana 병목 발견 (After 2)

HikariCP 대기 ~0: `decrementRanksAfter` DB bulk UPDATE 완전 제거로 커넥션 풀 압박 해소.

DELETE /api/v1/waitlists P95 ~3s → ~0s: 취소 시 DB 연산이 사라진 Sorted Set 전환 효과 직접 확인.

POST /auth/signup P95 ~6s: before()에서 300명 회원가입 HTTP 요청이 Grafana에 찍힌 것. 시나리오 본 플로우와 무관.

결과 해석

에러율 2.51% → 0%: 데드락 구조 자체를 제거해 HTTP 500 완전 소멸. 핵심 목표 달성.

P95 수치 증가는 에러 → 성공 전환 효과: Before/After 1에서 데드락으로 빠르게 실패한 요청들이 After 2에서는 전부 성공으로 완료됨 (총 요청 996 → 1052, +56건). 느린 요청도 P95에 포함되며 수치가 올라간 것이지 실제 성능 저하가 아님.

로컬 Docker 환경 기준: MySQL + Redis + Kafka + 앱 서버를 단일 머신에서 실행한 수치이므로 절대값보다 **개선율(에러율 0%, HikariCP ~0)** 위주로 해석해야 한다.



After 3 — Kafka 프로모션 전환 (2026-06-22)

변경 배경: After 2에서 ZSET(**ZADD**)이 **register()** 동기 경로에 포함되어 있었음. 친분 있는 시니어 개발자로부터 받은 피드백: "Redis는 캐싱에만 쓰고, 순서 보장은 Kafka 단일 파티션으로 처리하라."

변경 내용:

- WaitlistService** : **ZADD** / **ZRANK** / **ZREM** 전면 제거, **waitlist:zset:course:{courseId}** 키 삭제
- expireAndPromoteNext()** : **afterCommit**에서 **waitlist-promote-events** Kafka 발행
- WaitlistPromoteConsumer** (신규): 단일 파티션 소비 → DB **ORDER BY rank ASC** 로 첫 번째 WAITING 조회 → NOTIFIED 전환 + SSE 알림

- `EnrollmentCancelConsumer` : ZSET range 조회 → DB
`findFirstByCourse_IdAndWaitlistStatusAndDeletedAtIsNullOrderByRankAsc` 교체
- rank 값은 DB에 보존, `getMyWaitlists()` 조회 시 `w.getRank()` 직접 반환

항목	Before (DB)	After 1 (Redis)	After 2 (ZSET)	After 3 (Kafka)
전체 요청 수	996 (KO=25)	998 (KO=27)	1052 (KO=0)	1036 (KO=0)
평균 응답시간	1631ms	1895ms	2020ms	1654ms
P50	1317ms	1503ms	1577ms	1377ms
P75	2310ms	2745ms	3066ms	2499ms
P95	3266ms	4060ms	4693ms	3503ms
P99	4111ms	4874ms	5827ms	4289ms
처리량	142 req/s	124 req/s	116 req/s	148 req/s
HikariCP 대기	189	80	~0	~0
에러율	2.51% (25건)	2.71% (27건)	0%	0%

After 2 대비 After 3 개선율:

- P95: 4693ms → 3503ms (-25%)
- P99: 5827ms → 4289ms (-26%)
- 처리량: 116 → 148 req/s (+28%)



결과 해석:

- ZSET `ZADD` 를 동기 경로에서 제거하면서 P95 응답시간 단축 및 처리량 증가

- Kafka 단일 파티션이 순서 보장 담당 → Redis는 슬롯 카운터(`waitlist:course:{id}`) 역할만 수행
- 프로모션 로직이 Consumer로 완전히 분리되어 동기 경로 단순화, 에러율 0% / HikariCP ~0 유지
- HikariCP active=1 표시(`docker-compose down -v` 이후 클린 DB 상태)는 `before()` 단계 300명 계정 생성 구간에서 발생한 것으로 ZSET → Kafka 코드 변경과 무관

시나리오 3 — 스트레스 테스트 (StressTestSimulation)

개요

c6i.large 스펙(CPU 2 vCPU / Memory 4g) 제약 환경에서 수강신청 전체 플로우를 몇 명 동시 접속까지 처리할 수 있는지 Breaking point를 찾는다. Before/After 비교가 아닌 서버가 실제로 무너지는 시점 탐색이 목적.

5500개 계정을 JDBC 배치 INSERT로 사전 생성하고(HTTP 회원가입 대신 직접 INSERT — 테스트 시작 전 수초 내 완료), 점진적 주입 방식(`incrementUsersPerSec`)으로 부하를 단계별로 올린다.

주입 전략

단계	초당 주입	유지 시간	누적 주입
1	10/s	30s	~300명
2	20/s	30s	~900명
3	30/s	30s	~1800명
4	40/s	30s	~3000명
5	50/s	30s	~5100명

단계 사이 5초 램프업 포함. 총 주입 약 5100명.

API 호출 순서

`before()` 에서 5500명 계정 JDBC 배치 INSERT, `enrollment:course:3=9999`, `enrollment:course:4=9999` Redis 초기화 후 본 시나리오 실행.

1. `POST /api/v1/auth/login` — 개인 계정으로 로그인, accessToken 발급
2. `GET /api/v1/courses` — 강의 목록 조회

3. `POST /api/v1/enrollments` — UI/UX 강의(course_id=3) 수강신청 (201이면 다음 단계 진행)
4. `GET /api/v1/enrollments/me` — 3초 대기 후 내 수강신청 조회 (COMPLETED 상태 enrollmentId 추출)
5. `DELETE /api/v1/enrollments/{enrollmentId}` — 수강신청 취소 (enrollmentId 추출 성공 시만 진행)
6. `POST /api/v1/enrollments` — 수강 변경 (리더쉽, course_id=4)

단계별 통과 인원: 로그인(5100) → 강의조회(5100) → 수강신청(5100) → 내신청조회(5100) → 취소·재신청(1,103)

1~4단계를 마친 5100명 중 4000명이 5~6단계를 진행하지 못한 이유: 3초 대기 후 `/enrollments/me` 조회 시 Kafka Consumer가 아직 처리 완료하지 못한 것은 `COMPLETED` 상태가 아니어서 enrollmentId 추출 실패 → 해당 사용자는 이후 취소·재신청 단계 불가. 오류가 아닌 Kafka Consumer Lag 누적에 의한 정상 동작.

측정 결과 (5100명, c6i.large 시뮬레이션)

항목	스트레스 테스트
전체 요청 수	22,606 (KO=0)
P95	44,238ms
처리량	97 req/s
에러율	0%
Breaking Point	40/s 구간 (~3000명 동시)

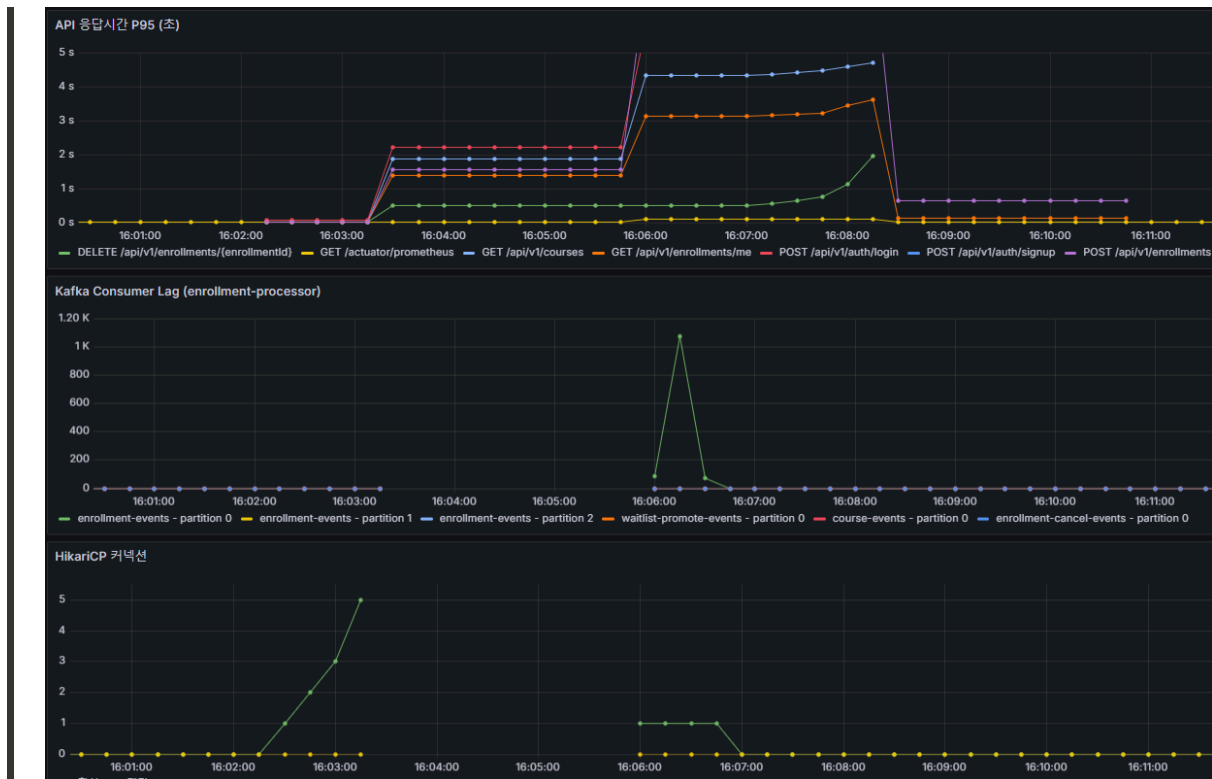
Breaking Point — 40/s 구간

40/s 주입 단계(3000번째 사용자 진입 시점)부터 P95 응답시간이 급격히 증가하여 44초에 도달.

원인 — BCrypt 로그인 CPU 포화:

BCrypt(cost=10)는 해시 연산당 약 200ms CPU를 점유한다. c6i.large(2코어) 기준 최대 처리량은 $2\text{코어} \times (1000\text{ms} / 200\text{ms}) = 10\text{ ops/s}$. 40/s 주입이 시작되면 BCrypt 처리 속도(10~20/s)를 초과하는 로그인 요청이 Tomcat 스레드를 계속 점유 → 스레드 풀 고갈 → 수강신청 API를 포함한 전체 요청이 큐잉 → P95 급등.

Grafana 모니터링



발견된 문제점

1. BCrypt 로그인 병목 (핵심)

수강신청 폭주 구간에서 로그인 요청이 동시에 몰리면 BCrypt CPU 연산이 Tomcat 스레드를 장시간 독점. 수강신청 자체는 Redis-only 동기 구간이라 빠르지만, 로그인이 스레드를 막아 전체 API 큐가 쌓임.

개선 방향: 로그인 전용 스레드 풀 분리 / BCrypt cost 값 낮춤(10 → 8) / 수강신청 기간 중 토큰 재발급 없이 30분 유지

2. Kafka Consumer Lag 1,200 누적

40/s 피크 구간에서 enrollment-events Consumer Lag이 최대 1,200까지 누적. Consumer 처리 속도가 발행 속도를 따라가지 못하는 구간 발생 → 3초 대기 후 조회 시 COMPLETED 상태 미반영 → 4,000명이 5~6단계를 진행 못함.

개선 방향: Consumer 병렬 처리 증가(파티션 수 확장) / 조회 재시도 로직 추가

3. Grafana 그래프 일부 구간 끊김

40/s 피크 구간에서 Prometheus scrape 간격(1s) 내에 응답하지 못한 메트릭 누락으로 그래프 선이 끊어짐. 서버 장애가 아닌 부하 집중 구간의 수집 지연.

잘된 점

HikariCP 최대 5개 — Redis-only 설계 검증

5100명이 동시 수강신청하는 상황에서도 HikariCP 활성 커넥션이 최대 5개로 유지됨. 동기 구간(enroll)을 Redis-only로 전환한 설계가 DB 병목을 차단했음을 실측으로 확인. 시나리오 1 Before 측정의 HikariCP 대기 180개와 대비되는 결과.

최적화 후 재측정 (2026-06-24)

적용된 최적화 항목

- Kafka Consumer concurrency: 1 → 3 (파티션 3개와 스레드 수 일치)
- HikariCP maximum-pool-size: 10 → 20
- BCrypt cost: 10 → 8

2×2 측정 비교

항목	① cost10 + show_sql on	② cost10 + show_sql off	③ cost8 + show_sql off	④ cost8 + show_sql on
KO	9	0	0	0
P95	49,010ms	46,021ms	21ms	49ms
P99	54,848ms	51,073ms	71ms	94ms
Mean	13,936ms	13,236ms	9ms	12ms
Max	60,016ms	57,845ms	557ms	447ms
처리량	93.85 req/s	95.06 req/s	175.86 req/s	175.86 req/s

①②는 Kafka concurrency=3, HikariCP=20 적용 후 측정한 값. ①②가 ③④보다 나쁜 이유는 BCrypt cost10이 주 병목이었기 때문. ② 초기 측정(KO 476건)은 이상값으로 재측정 후 교체.

결론

- **핵심 병목은 BCrypt cost10**: cost 8로 낮추는 것만으로 P95 ~49,000ms → 21ms 달성
- **show_sql 영향은 부가적**: off(21ms) vs on(49ms) — 2.3배 차이지만 둘 다 서비스 가능 수준
- show_sql이 기존에 큰 병목처럼 보인 이유: cost10으로 로그인 느려진 상태에서 SQL 로그 I/O가 누적되어 함께 폭발한 것. cost8 환경에서는 영향 미미

BCrypt cost 10 → 8 판단 근거

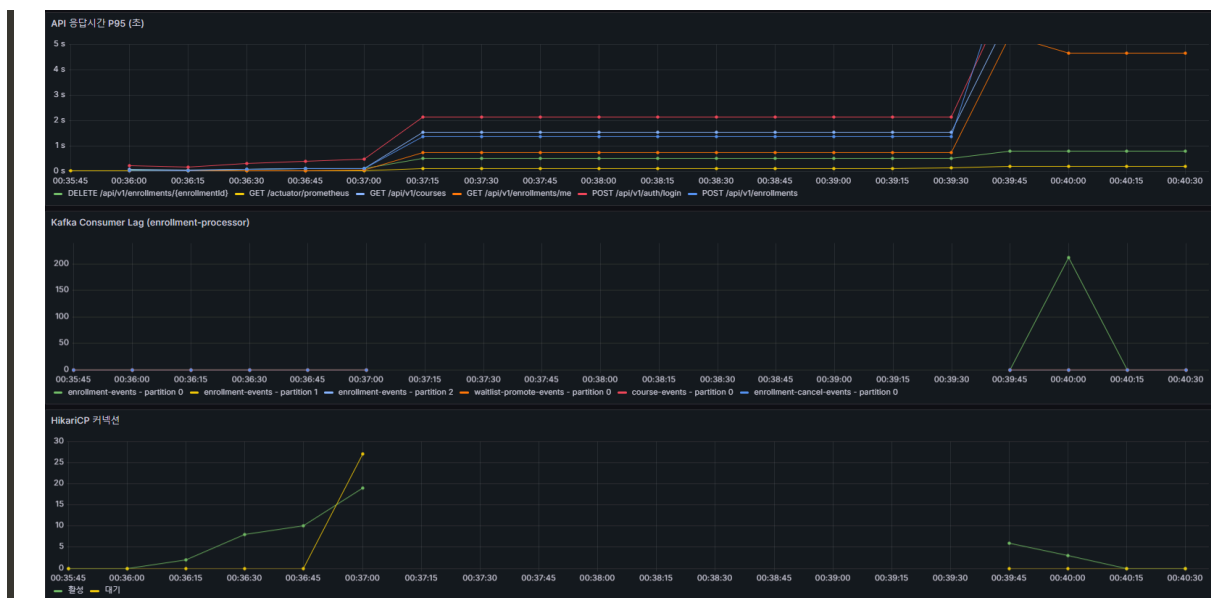
cost 10 → 8은 상시 적용이 아니라 수강신청 폭주 기간에만 한정한 임시 조치다. BCrypt cost는 1 증가할 때마다 연산량이 2배가 되므로, cost 10과 8은 연산 시간이 정확히 4배

차이 난다. 이는 DB 유출 시 오프라인 브루트포스 저항력이 cost 8에서 1/4로 낮아진다는 의미이지만, 적용 구간을 수강신청 기간(연 2회, 각 수 시간)으로 제한하면 실제 노출 시간은 극히 제한적이다. 코드상 기본값(`security.bcrypt.strength`)은 10이며, cost 8은 이 폭주 기간 동안만 환경변수로 전환하는 것을 전제로 한다.

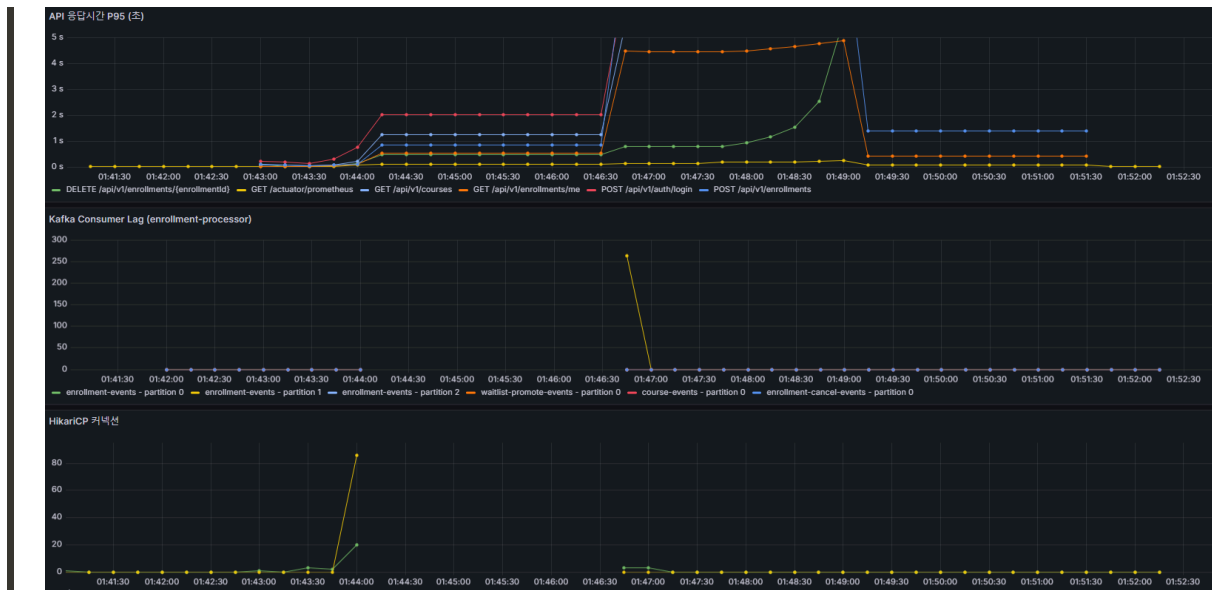
최종 수치 — Before → After

지표	Before (cost10 + show_sql on)	After (cost8 + show_sql off)	개선율
P95	44,238ms	21ms	-99.95%
P99	—	71ms	—
KO	0	0	—
처리량	97 req/s	175.86 req/s	+81%

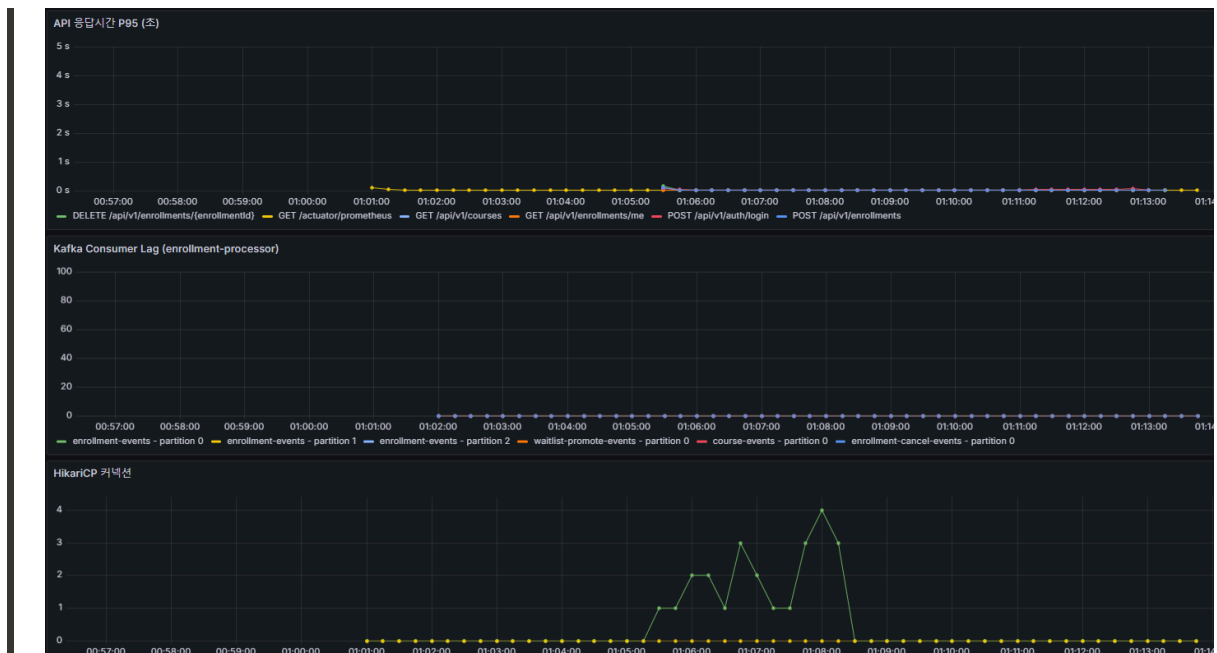
Grafana — ① cost10 + show_sql on (Before)



Grafana — ② cost10 + show_sql off



Grafana — ③ cost8 + show_sql off (After)



Grafana — ④ cost8 + show_sql on

